

Commitments, Dispositions and Traces: An Ontological Account of Accountable Delegation in Multi-Agent AI Systems

[authors]

[institute]

Abstract. [150 words. Three open questions left by AI-Intent (ER 2026): accountable to whom, what a disposition is formally, and whether an auditor can query a trace without reading code. UFO-C commitments and claims, UFO dispositions and a gUFO export answer them. Reference implementation; evaluation E1 to E4; two corrections to the earlier grounding.]

Keywords: Conceptual modeling · UFO · Multi-agent systems · Accountability · Delegation · Dispositions

1 Introduction

[Recall AI-Intent in four sentences: Mandate, Intent Enforcement, Accountability Trace; Compliance Agent as a gate; zero forced-pass over 190 evaluations.]

[State the three questions the ER 2026 paper leaves open.]

1. Accountable to whom? The trace records events; the model has no normative relation between delegator and delegatee.
2. What is a disposition? Presets are prompt text and a score; manifestation is undefined.
3. Can a regulator query a trace? Integrity conditions live in scoring code.

[Claim: UFO-C, UFO-A/B and gUFO answer them, and the answers are implementable. Contributions C1 to C4 as in PLAN.md. Paper outline.]

2 Background

2.1 UFO in the extent needed

[Endurants and perdurants; modes inhere in endurants; extrinsic modes externally depend on other individuals; relators mediate; events are non-repeatable and bring about situations; dispositions are intrinsic modes manifested in events when a triggering situation obtains. UFO-C: social commitments and claims, relators constituted by them. gUFO as the OWL rendering: classes and properties used here (`gufo:Relator`, `gufo:ExtrinsicMode`, `gufo:IntrinsicMode`, `gufo:Event`, `gufo:Situation`, `gufo:inheritsIn`, `gufo:externallyDependsOn`, `gufo:mediates`, `gufo:historicallyDependsOn`, `gufo:manifestedIn`, `gufo:broughtAbout`).]

2.2 AI-Intent

[The nine constructs and their ER 2026 grounding (table). Note what the present paper changes: the Compliance Verdict row and the Src column, see Section 8.]

3 Delegation as Commitment and Claim

AI-Intent represents delegation structurally: a Mandate names its parent Mandate, and the Principal names the Mandates it owns. The structure says which Mandate refines which; it does not say that anyone owes anything to anyone. UFO-C supplies the missing normative relation. In UFO-C, a social relator is constituted by a commitment of one party towards another and the corresponding claim of the second party against the first. A commitment is an extrinsic mode: it inheres in the committed party and is existentially dependent on the party it is directed to. We apply this pattern to delegation.

Definition 1 (Delegation). *Let p be a Principal or an Agent, a an Agent, and M_a the Mandate of a . A Delegation $D(p, a, M_a)$ is a relator that mediates p and a and is constituted by*

- a Commitment $C(a \rightarrow p, M_a)$: *an extrinsic mode inhering in a and externally dependent on p , whose content is that a acts only within the intent scope of M_a and never violates a boundary constraint $\langle \text{text}_i, \phi_i, \tau_i \rangle$ of M_a ; and*
- a Claim $K(p \rightarrow a, M_a)$: *an extrinsic mode inhering in p and externally dependent on a , whose content is that p is entitled to outputs of a that satisfy M_a and to a verdict that names every breach.*

The rule identifiers that operationalise C are the rules of the registry that apply to a ; we write $R(C)$ for this set.

The commitment carries the Mandate’s content, the claim carries the entitlement to enforcement. Neither exists without the other, and neither exists without both parties, which is what distinguishes a social relator from a mere attribute of the delegatee.

The chain. In the reference implementation the Principal delegates to the orchestrator (**central**) and the orchestrator delegates to the three specialists. Each sub-delegation records its parent, so the chain is a tree rooted at the Principal:

$$D(\text{principal}, \text{central}, M_c) \succ D(\text{central}, a, M_a) \quad \text{for } a \in \{\text{stocks}, \text{bonds}, \text{materials}\}.$$

The chain is built from the Mandate registry at the start of every session and logged as the event **delegation.establish** before any sub-agent is called. A Delegation is therefore never implicit: if it is not in the trace, it did not govern the session.

Containment. A delegator cannot grant more than it holds. We state this as a condition on the chain that is decidable at design time.

Definition 2 (Mandate containment). *Let $D(p, a, M_a)$ have parent $D(q, p, M_p)$ with p an Agent. M_a is contained in M_p if (i) M_a names M_p as its parent, (ii) both are owned by the same Principal, and (iii) for every bounded risk parameter r of M_a with governing parameter r' of M_p , $M_a(r) \leq M_p(r')$.*

Condition (iii) uses an explicit map from sub-mandate parameters to parent parameters. In the investment domain the map binds the specialists' allocation caps to the orchestrator's cap on any single asset class: a stocks position cap of 10% and a materials cap of 15% are both contained in the orchestrator's 40%, whereas a materials cap raised to 55% is not. The implementation evaluates the condition in `check_chain_containment` and logs the result with the chain; a violated containment marks the `delegation.establish` event with status `constraint_violation` before the first message is sent. Evaluation E1 (Section 6) mutates each bounded parameter above its parent bound and confirms that every mutation is reported.

Accountability resolution. The ER 2026 Compliance Agent drops a message that cannot be made compliant within the revision budget and records a *forced block*. With Definition 1 in place, a forced block has a normative reading: the blocked agent has breached its Commitment.

Definition 3 (Commitment breach). *A forced block of agent a in session s with violated rules V brings about a situation Commitment Breach $B(a, s, V)$ in which $C(a \rightarrow p, M_a)$ is unfulfilled. The parties answerable for B are the delegators along the path from $D(p, a, M_a)$ to the root: p , the delegator of p , and so on up to the Principal.*

The orchestrator resolves the path from the logged chain and records it as `delegation.breach.{a}` together with V and with $V \cap R(C)$, the violated rules the agent had actually committed to. The last intersection is not idle: Section 6 reports that it exposed five rules the gate applied without their being part of any commitment.

4 Dispositions and Their Manifestation

The ER 2026 evaluation stressed the gate with four disposition presets (neutral, aggressive broker, reckless portfolio manager, groupthink) and measured Disposition Containment as a score. In the implementation a preset is a vector of five numbers per agent and a sentence prepended to the system prompt. The model said nothing about what a disposition is or what it means for one to show.

UFO has an account. A disposition is an intrinsic mode of its bearer that is manifested in an event when a triggering situation obtains; it may exist without ever being manifested. We adopt this account with the five preset dimensions as kinds.

Definition 4 (Disposition). A Disposition is a tuple $\delta = \langle k, a, g, S_k, R_k \rangle$ with kind $k \in \{\text{self_serving}, \text{risk_seeking}, \text{overconfident}, \text{anti_customer}, \text{conformist}\}$, bearer a an Agent, degree $g \in [0, 1]$, a triggering situation S_k described as a class of Proposed Actions, and a characteristic rule set R_k drawn from the registry. An agent bears one Disposition per kind with positive degree.

Definition 5 (Manifestation). Let e be a compliance rejection or block event in session s that targets agent a and names violated rules V_e . e is a manifestation of $\delta = \langle k, a, g, S_k, R_k \rangle$ if $g \geq \theta$ and $V_e \cap R_k \neq \emptyset$. The attributed rules of the manifestation are $V_e \cap R_k$.

The threshold θ is 0.5 in the implementation; the neutral preset’s degrees of 0.1 lie below it, so under the neutral preset no rejection is ever attributed to a disposition. The characteristic rule sets encode what each kind of pressure is expected to produce: a risk-seeking agent breaches caps and leverage prohibitions, an overconfident one omits required disclosures, a conformist one follows another agent out of its own scope. Table ?? lists the sets. [table from MANIFESTATION_MAP with one-line justification per kind]

Attribution is deterministic and reads only the log: after the sub-agent calls complete, the orchestrator scans the session’s rejection and block events, applies Definition 5 for every Disposition the agent bears, and records the result as `disposition.manifest.{a}`. A rejection may manifest several dispositions at once, which is as it should be; a cap breach by a reckless agent is both risk-seeking and self-serving.

Disposition Containment restated. With these definitions the ER 2026 metric becomes a statement about dispositions and the gate. The gate *contains* a disposition δ in session s if every manifestation of δ in s is a rejection or a block, and no manifestation is a delivery. The zero-forced-pass result of ER 2026 is then the statement that across 190 sessions no manifestation of any disposition reached delivery. The present paper adds what the earlier one could not say: which dispositions were manifested, how often, and in which rules.

5 Traces as Situations and Integrity Queries

The ER 2026 paper grounds the Accountability Trace as a situation and the Log Entry as an event, and leaves it there. The grounding becomes useful when the trace is exported in a form that carries it. We export every session as an RDF graph typed with gUFO, the OWL rendering of UFO, and state the framework’s invariants as SPARQL queries over that graph.

Export mapping. Table 1 gives the mapping. Three decisions deserve comment.

First, the Compliance Verdict is a mode of the *agent*, not of the Proposed Action. Modes inhere in endurants, and a Proposed Action is an event; the ER 2026 table said otherwise and is corrected here. The verdict’s dependence on the action it evaluated is historical, and the export records it with `gufo:historicallyDependsOn`.

Table 1. Export of a session to gUFO. Every individual is created from the MCP log alone.

Construct	gUFO typing	Source in the log
Session	<code>gufo:Event</code>	the session id
Log Entry	<code>gufo:Event</code> , proper part of the Session	every message
Proposed Action	<code>gufo:Event</code>	<code>intent.route</code> , <code>{a}.result</code> , <code>intent.synthesize</code>
Compliance Verdict	<code>gufo:IntrinsicMode</code> inhering in the target agent, <code>historicallyDependsOn</code> the evaluated Proposed Action	<code>in-compliance.approve.*</code> , <code>compliance.reject.*</code> , <code>compliance.block.*</code>
Accountability Trace	<code>gufo:Situation</code> brought about by the Session, comprising every Log Entry	derived
Delegation	<code>gufo:Relator</code> mediating delegator and delegatee	<code>delegation.establish</code>
Commitment, Claim	<code>gufo:ExtrinsicMode</code> , inhering in one party, externally dependent on the other	<code>delegation.establish</code>
Disposition	<code>gufo:IntrinsicMode</code> with de- gree, <code>manifestedIn</code> a rejection event	<code>disposition.active</code> , <code>disposition.manifest.*</code>
Commitment Breach	<code>gufo:Situation</code> brought about by the breach event	<code>delegation.breach.*</code>
Principal, Agent	<code>gufo:FunctionalComplex</code>	message endpoints
Mandate	<code>gufo:Object</code> (see Section 8)	<code>delegation.establish</code>

To make that link exact rather than inferred, the Compliance Agent now writes the log id of the evaluated action into every verdict.

Second, the export reads nothing but the log. Delegations, dispositions and manifestations appear in the graph only if the orchestrator logged them. This keeps the invariant of ER 2026 that the log is the source of truth, and it has a consequence the evaluation exploits: a session in which no commitment was logged is a session whose breaches have no answerable party, and the queries say so.

Third, gUFO has no class for normative descriptions, so the Mandate is typed as `gufo:Object`. The typing is weak and we say so; Section 8 discusses it.

Integrity queries. Eight invariants of the framework are stated as SPARQL queries whose result set must be empty (Table 2). Each row returned is a violation with the individuals involved. The queries are short; Q2, the restatement of the containment claim, is shown in full.

Listing 1.1. Q2: no delivery after a block. Entries carry a sequence number in log order.

Table 2. Integrity checks. Each is a SPARQL query over the session graph; a returned row is a violation.

Id Invariant

- Q1 A Compliance Verdict historically depends on exactly one Proposed Action.
 Q2 After a block of agent *a* no final approval of *a* occurs in the same session.
 Q3 Every blocked agent bears a Commitment, so its breach has an answerable party.
 Q4 Every Commitment Breach is answerable to the session’s Principal.
 Q5 A Disposition is manifested only in events targeting its bearer, and only with positive degree.
 Q6 Every Proposed Action at the analysis checkpoint receives a verdict.
 Q7 The Accountability Trace comprises every Log Entry of the session.
 Q8 Every rule a verdict names against *a* belongs to the Commitment of *a*.
-

```
SELECT ?agent ?block ?final WHERE {
  ?block a aii:LogEntry ; aii:method ?bm ; aii:targetAgent ?
    agent ; aii:sequence ?sb .
  FILTER(STRSTARTS(STR(?bm), "compliance.block."))
  ?final a aii:LogEntry ; aii:method ?fm ; aii:targetAgent ?
    agent ; aii:sequence ?sf .
  FILTER(STRSTARTS(STR(?fm), "compliance.approve.") &&
    STRENDS(STR(?fm), ".final"))
  FILTER(?sf > ?sb)
}
```

Q2 is the claim that gives AI-Intent its name for bounded autonomy, now readable by an auditor who has never seen the Python code. Q3, Q4 and Q8 are the claims of Section 3; Q5 is the claim of Section 4; Q1, Q6 and Q7 are the structural conditions on which the others rest.

6 Reference Implementation and Evaluation

[Modules and hooks (PLAN.md Section 4). E1 containment; E2 attribution agreement over TC-16 to TC-19; E3 invariants over regenerated sessions, with the 21 surviving archived sessions as the before-hooks comparison (e3-archive-checks.md); E4 cost: triples per session and query time.]

[Finding from Q8: five disposition-integrity rules applied by the gate were absent from the registry and hence from every commitment; registered after the finding. State this as an example of the checks catching a specification gap that the ER 2026 evaluation could not see.]

[Results tables from paper2/e2-e3-results.md. Threats to validity: rule-set attribution is a modelling decision; model dependence; the regenerated sessions are not the ER 2026 sessions.]

7 Related Work

[UFO-C commitments and claims; service ontologies built on them. Delegation in Tropos and GAIA. Norm representations: LegalRuleML, ODRL. Provenance: PROV-O as the nearest standard to the trace export; argue what gUFO adds (modes, relators, dispositions) and what PROV-O adds (tooling), and whether to align.]

8 Corrections to the Earlier Grounding

[(a) Verdict as mode of an event is ill-formed; corrected reading (Section 5). (b) Derived versus design: all nine assignments are choices; the useful distinction is whether the category fixes the construct's formal behaviour completely. (c) Intent Scope as sortal universal: a scope string carries no identity principle; move to the second group or argue the sortal reading. (d) Mandate has no gUFO class; propose `aii:Mandate` with a documented reading as normative description.]

9 Conclusion

[Answers to RQ1 to RQ3. What UFO contributes beyond categorisation. Future work: PROV-O alignment, defeasible commitments, Mandate as normative description.]

References